Technical Report: Policy-as-Code for Digital Sovereignty – Limitations and Recommendations

ATUL0012345 · Student L00205332 · HCP Terraform (Free) + Sentinel, AWS `eu-west-1` / `eu-central-1` · Repository: `L00205332/ATUL0012345-sentinel-sovereignty` · AI use declared in `prompts.md`

Outcome. Six Sentinel policies (sovereignty, network, encryption, IMDSv2, supply chain: hard-mandatory, bundled; instance types: advisory) were deployed to HCP Terraform and verified with 57 unit tests, 53 bundle tests, mutation tests and seven live scenarios. Every non-compliant pull request (an off-EU backup copy, an unencrypted volume, IMDSv1, an unpinned module) was **blocked at plan time, before any AWS API call**. The compliant stack (Ireland, with an encrypted Frankfurt DR copy) was applied, and the advisory policy warned without blocking.

Limitations found

#	Limitation	Evidence
1	**Blind outside Terraform.** A manual CLI copy of sensitive data to `us-east-1` passed every policy (refresh-only `+0 ~0 -0`) and was found only by the out-of-band audit. The copy also lost the organisation's customer-managed key.	Scenario 6
2	**Region ≠ jurisdiction.** EU regions of a US-headquartered provider remain exposed to the US CLOUD Act. Global services (IAM, CloudFront) and region-less targets (S3 replication ARNs) can only be exempted, and residency depends on correct `data_classification` tags.	§7.2
3	**Attributes, not consequences.** No policy flagged that an unpinned "latest" AMI made a routine change **replace the sensitive data volume**. Values unknown until apply (KMS keys, inherited encryption) had to be checked indirectly.	Scenario 5
4	**Mocks are not the platform.** HCP passes plan numbers as floats, mocks store ints: a rule that passed 40 local tests failed every HCP run.	PR #13
5	**The gate is the run, not the merge.** GitHub allowed merging a PR with a failed check; an advisory failure shows a green check.	PRs #12, #17
6	**Supply-chain blind spots.** Sentinel can't verify provider binaries, module internals or maintainer/registry compromise; modules aren't in the lock file.	§4.2, PR #21
7	**Platform limits shape the design.** HCP Free allows one policy set, no VCS-linked policy sets and one mandatory policy, which forced API deployment, bundling and a single enforcement level. Deployed policies can drift from `main`.	§6.2

Recommendations

1. **Guardrails for every identity, not just the pipeline:** an AWS Organizations SCP denying non-EU regions, complementing the region-restricted HCP IAM user.
2. **Continuous detection:** AWS Config rules and CloudTrail alerts (e.g. `CopySnapshot`, `CreateReplicationConfiguration`) instead of a manually run audit.
3. **Make the PR gate binding:** branch protection on `main` requiring the HCP and CI checks; review advisory results explicitly.
4. **Protect data from destructive plans:** a hard-mandatory rule blocking deletion or replacement of `data_classification = sensitive` resources unless explicitly approved; pin images (`ignore_changes = [ami]` or a fixed AMI ID).
5. **Replace static keys with OIDC dynamic credentials** for HCP, removing the long-lived secret held by a SaaS provider.
6. **Strengthen the supply chain:** keep exact module pins and the multi-platform lock file, review module versions before approval, consider a private registry, and keep CI actions SHA-pinned.
7. **Test against the real platform:** keep real-plan regression mocks, refresh them after every provider or Terraform upgrade, and keep at least one live PR test per policy area.
8. **Run policies as a product:** clear ownership, versioned releases, automated deployment (CI `deploy-policies` with `TFE_TOKEN`) and a review on every provider major version or platform change. A paid HCP tier would remove the bundling and single-level constraints.
9. **Address jurisdiction beyond regions:** customer-held keys with restrictive key policies and, where legal reach matters, contractual controls or an EU-operated cloud.

Conclusion. Policy-as-Code is an effective, auditable **first line** for digital sovereignty: it prevents non-compliant infrastructure before it exists. It is not sufficient on its own. Sovereignty needs it combined with account-wide guardrails, continuous detection and controls on the jurisdiction of the provider itself.